

JWT 인증과 Supabase Auth 회원가입·로그인

1. JWT란 무엇인가

JWT(JSON Web Token)는 서버가 발급하는 **서명된 토큰**입니다.

토큰 형태

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJ1c2VyX2lkIjoiaWJjIiwiaXhwIjoxNzAwMDAwMDAwfQ
.Sf1KxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

. 으로 구분된 **세 부분**으로 이루어집니다.

부분	내용	비고
헤더 (Header)	{"alg": "HS256", "typ": "JWT"}	알고리즘 정보
페이로드 (Payload)	{"sub": "user-id", "exp": 1700000}	사용자 정보 + 만료 시간
서명 (Signature)	HMAC(헤더 + 페이로드, 비밀키)	위변조 방지

XXXXXX.YYYYYY.ZZZZZZ

헤더(Header)

내용(Payload)

서명(Signature)

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MzkwMjQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

헤더(Header)	내용(Payload)	서명(Signature)
<pre>{ "alg": "HS256", "typ": "JWT" }</pre>	<pre>{ "sub": "1234567890", "name": "John Doe", "iat": 1516239022 }</pre>	<pre>HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), your-256-bit-secret)</pre>

페이로드는 Base64 인코딩 — 누구나 볼 수 있음

```
import base64, json

payload = "eyJ1c2VyX2lkIjoiaWJjIiwiaXhwIjoxNzAwMDAwMDAwfQ"
decoded = base64.b64decode(payload + "==")
print(json.loads(decoded))
# {"user_id": "abc", "exp": 1700000000}
```

페이로드는 누구나 디코딩할 수 있습니다.

민감한 정보(비밀번호, 카드번호)는 절대 페이로드에 넣지 않습니다.

왜 안전한가 — 서명 검증

서버가 발급할 때 :

서명 = HMAC(헤더.페이로드, 서버만 아는 비밀키)

클라이언트가 토큰을 받아서 페이로드를 수정하면 :

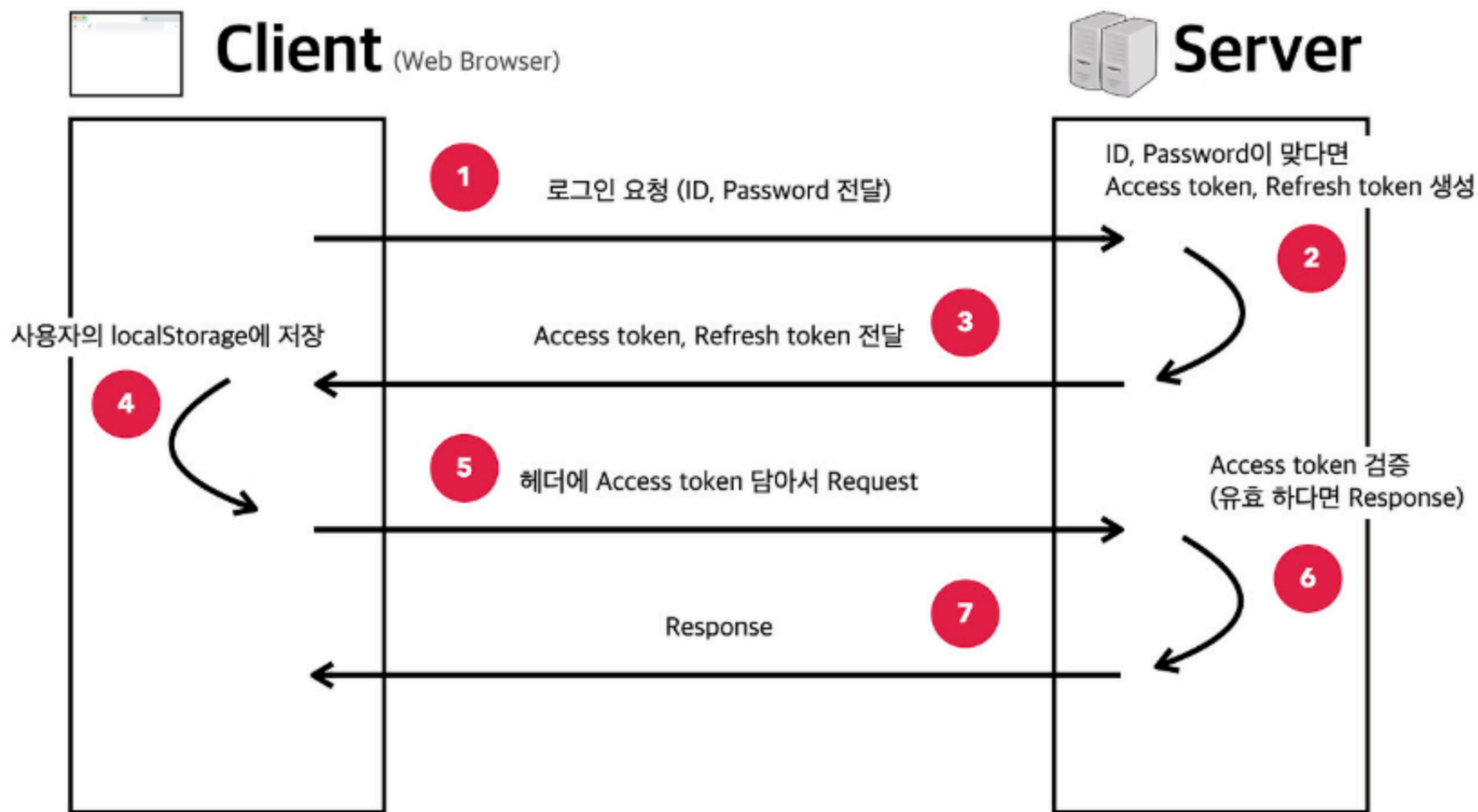
수정된 토큰의 서명 $\neq$ 서버 검증 결과

→ "위변조된 토큰" → **401** 거부

비밀키를 모르면 서명을 만들 수 없음

→ 페이로드를 바꾸면 서명이 맞지 않아 거부됨

JWT 동작 방식



JWT 동작 방식 (이미지 출처 : <https://joie-kim.github.io/JWT-Auth/>)

2. access_token vs refresh_token

Supabase Auth 로그인은 두 가지 토큰을 반환합니다.

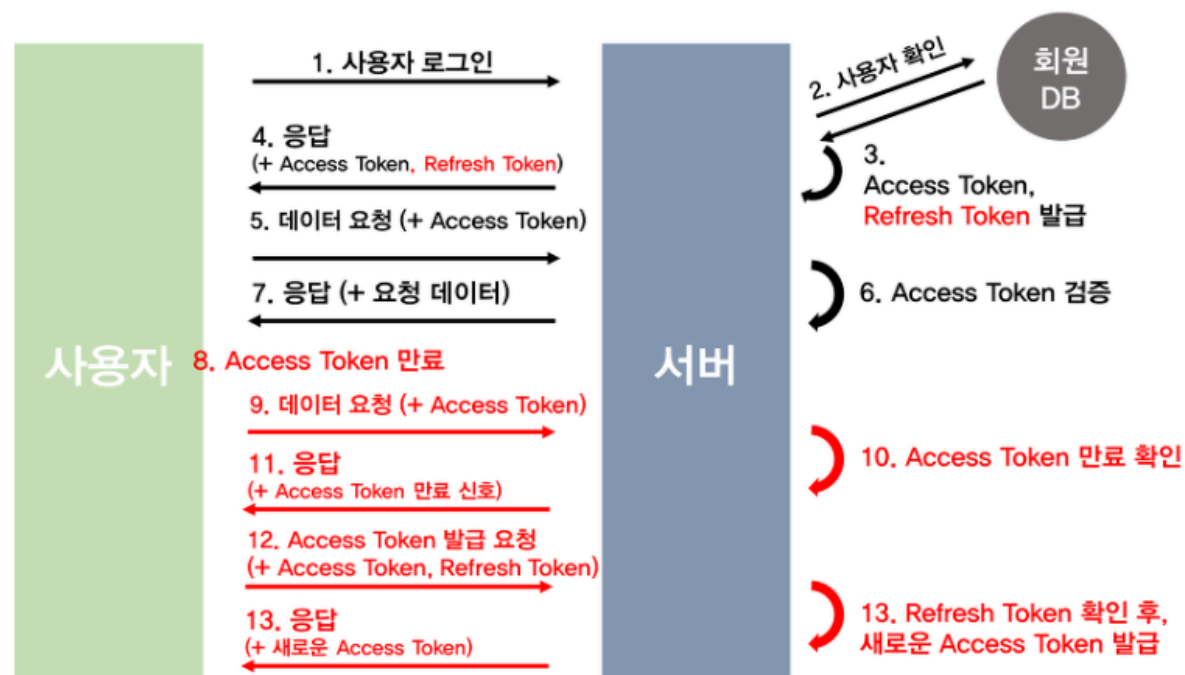
항목	access_token	refresh_token
용도	API 요청 인증	access_token 갱신
전달 방법	Authorization: Bearer {token} 헤더	서버 내부에만 보관
만료 시간	짧음 (기본 1시간)	김 (기본 7일)
유출 시	1시간 후 자동 만료	즉시 무효화 조치 필요
저장 위치	메모리 또는 session	HttpOnly 쿠키 또는 서버

전체 프로세스

로그인 → access_token (1시간) + refresh_token (7일)

1시간 후: access_token 만료
↓
refresh_token으로 새 access_token 발급 (재로그인 불필요)

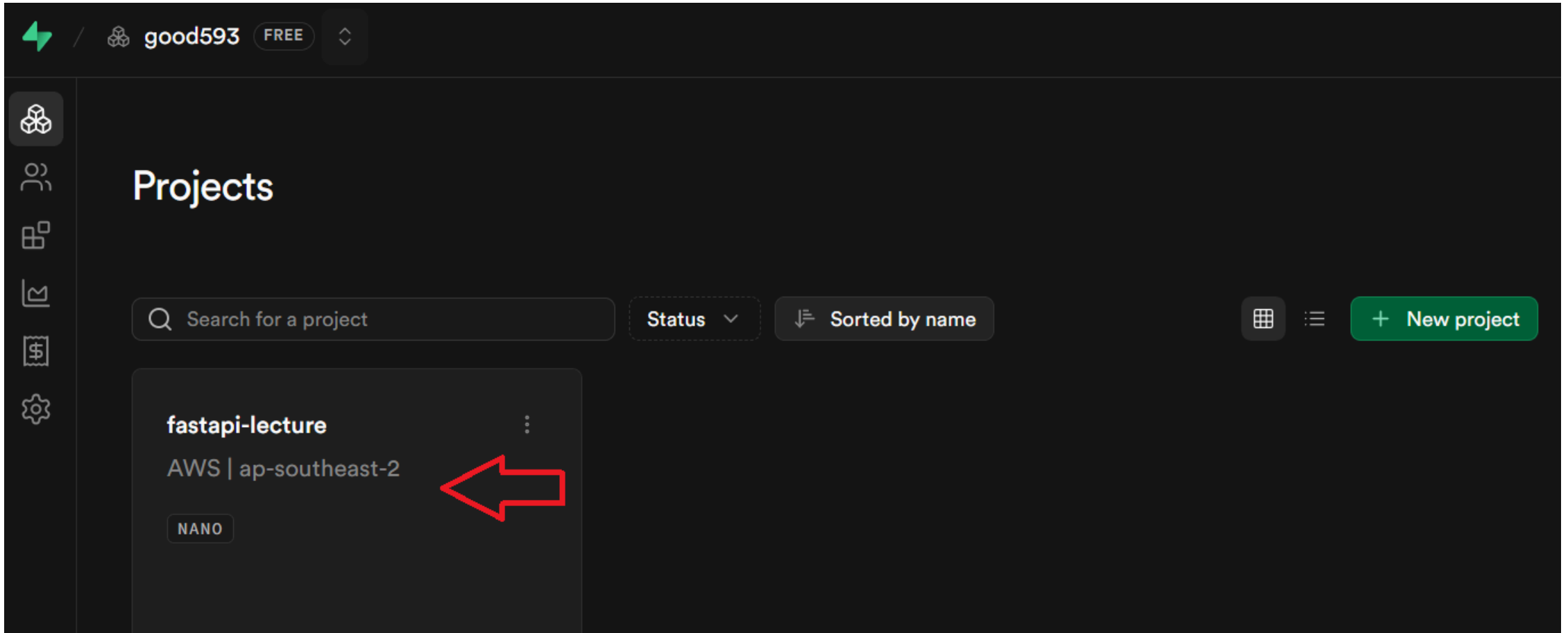
7일 후: refresh_token 만료
↓
다시 이메일/비밀번호로 로그인



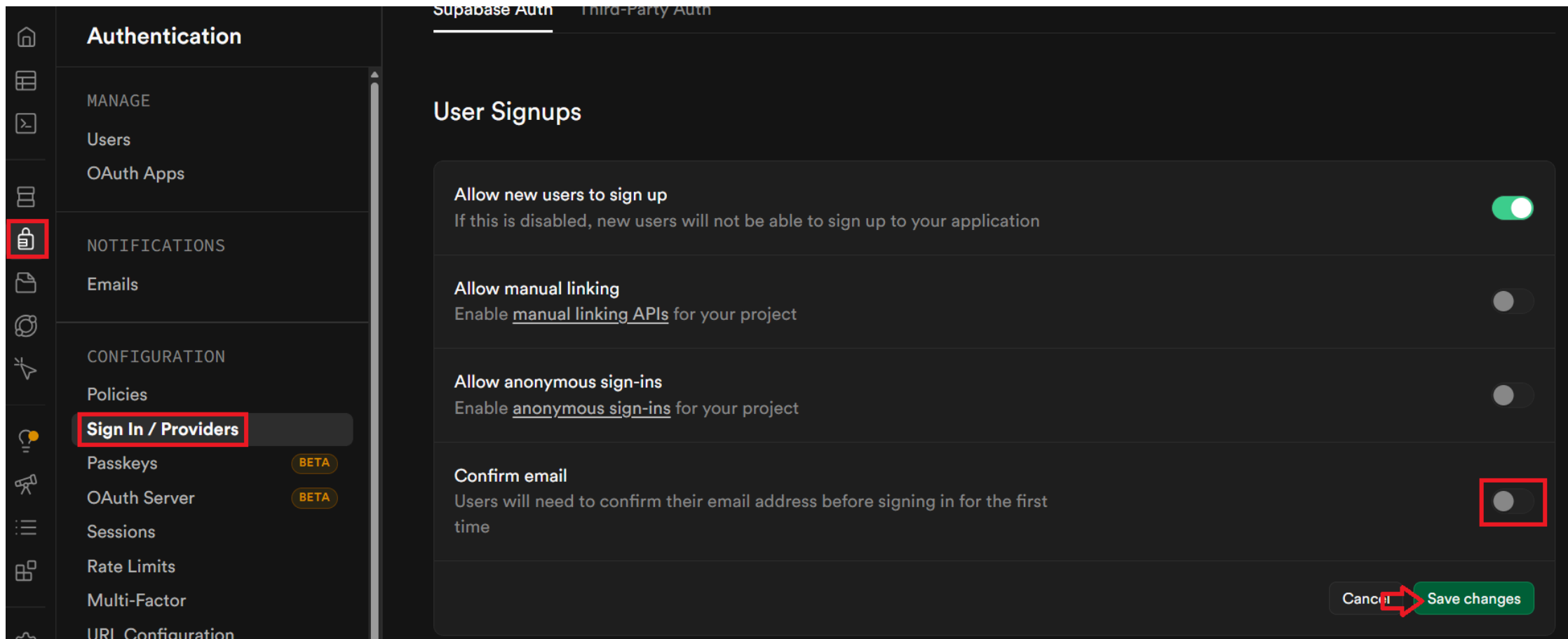
3. Supabase Auth 설정

이메일 인증 비활성화 (개발용)

1. Supabase Dashboard에서 프로젝트 선택



2. Authentication → Sign In / Providers → "Confirm email" 옵션을 끄기 (OFF)



이 설정을 끄면 회원가입 즉시 로그인할 수 있습니다.
운영 환경에서는 반드시 켜야 합니다.

4. 회원가입 구현

```
from pydantic import BaseModel, EmailStr, Field
from supabase import create_client

def public_client():
    return create_client(SUPABASE_URL, SUPABASE_ANON_KEY)

@app.post("/auth/signup", status_code=201)
def signup(data: SignupRequest):
    try:
        result = public_client().auth.sign_up({
            "email": data.email,
            "password": data.password,
        })
    except Exception as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    ...
```

EmailStr — 이메일 형식 자동 검증

```
# pydantic[email] 설치 필요
# requirements.txt에 email-validator 포함

from pydantic import EmailStr

class SignupRequest(BaseModel):
    email: EmailStr # "user@" → 422, "user@example.com" → 통과
```

5. 로그인 구현

응답에서 `access_token` 을 복사해서 이후 요청에 사용합니다.

```
@app.post("/auth/login")
def login(data: LoginRequest):
    try:
        result = public_client().auth.sign_in_with_password({
            "email": data.email,
            "password": data.password,
        })
    except Exception as exc:
        raise HTTPException(status_code=401, detail="로그인에 실패했습니다") from exc

    return {
        ...
        "access_token": result.session.access_token,
        "refresh_token": result.session.refresh_token,
        "token_type": "bearer",
    }
```

6. HTTPBearer — Bearer 토큰 받기

헤더 형식

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Bearer 다음에 access_token을 붙입니다. (공백 포함, 대소문자 중요)

FastAPI HTTPBearer 설정

```
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from fastapi import Depends, HTTPException
from pydantic import BaseModel

security = HTTPBearer()

class CurrentUser(BaseModel):
    id: str
    email: str

def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security),
) -> CurrentUser:
    token = credentials.credentials # "Bearer " 이후의 실제 토큰값

    try:
        result = public_client().auth.get_user(token)
    except Exception as exc:
        raise HTTPException(status_code=401, detail="유효하지 않은 토큰입니다") from exc

...
```

보호된 엔드포인트

```
@app.get("/me")
def get_me(current: CurrentUser = Depends(get_current_user)):
    return {
        "user_id": current.id,
        "email": current.email,
        "message": "인증 성공!"
    }
```

7. Swagger에서 Bearer 토큰 입력 방법

1. POST /auth/login 실행 → 응답에서 "access_token" 값 복사
2. Swagger UI 우측 상단 Authorize 버튼 클릭
3. Value 입력란에 입력:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
(로그인 응답의 access_token 값만 입력)
4. Authorize 버튼 클릭 → Close
5. 이후 모든 보호된 엔드포인트 요청에 자동으로 헤더가 추가됨

Swagger UI가 Bearer 접두사를 자동으로 붙입니다.

입력란에 Bearer eyJ... 를 넣으면 접두사가 중복되어 인증에 실패합니다.

8. Swagger에서 토큰 없이 요청하면

HTTPBearer 가 설정된 엔드포인트에 토큰 없이 요청하면:

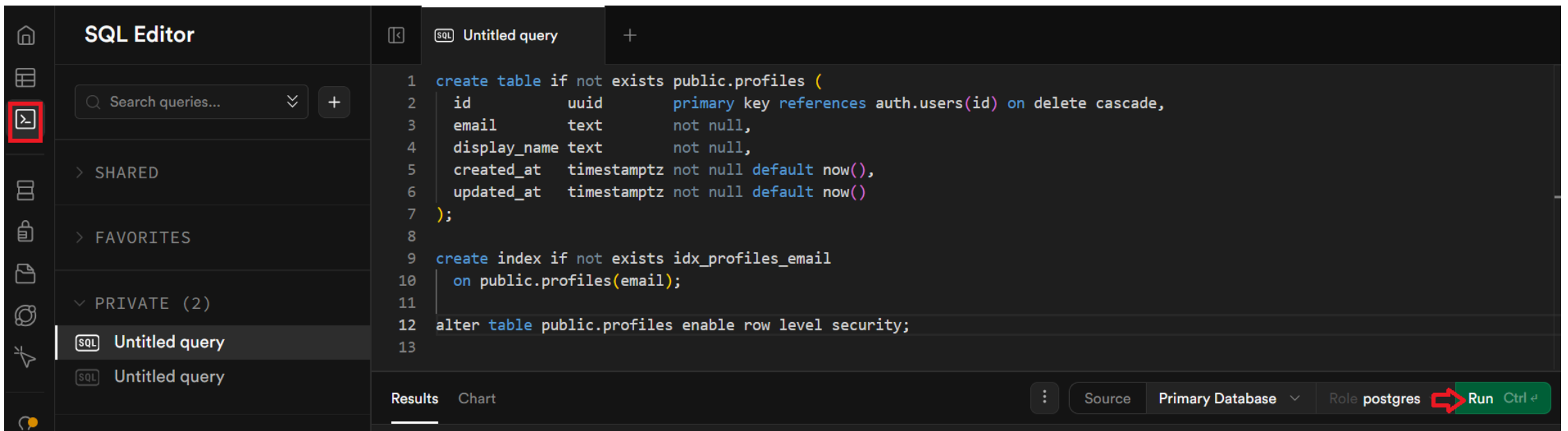
```
// 403 Forbidden
{
  "detail": "Not authenticated"
}
```

FastAPI HTTPBearer 가 요청을 엔드포인트에 전달하기 전에 인증 정보가 없는 요청을 거부합니다.

Swagger에서 자물쇠 아이콘이 닫혀 있으면 토큰이 설정된 상태입니다.

9. 서버 실행 및 Swagger 실습

schemam.sql을 이용해서 Supabase에 테이블 생성하기



생성된 테이블(profiles) 확인

The screenshot displays the 'Table Editor' interface for a database. On the left, a sidebar lists tables in the 'public' schema: 'app_users', 'conversations', 'messages', and 'profiles'. The 'profiles' table is selected. The main area shows the table's structure with columns: 'id' (uuid), 'email' (text), 'display_name' (text), and 'created_at' (timestampz). The table is currently empty, as indicated by the message 'This table is empty' at the bottom. The interface includes a search bar, a filter bar, and an 'Insert' button.

id	email	display_name	created_at
----	-------	--------------	------------

SUPABASE_ANON_KEY 추가

Settings

CONFIGURATION

General

Compute and Disk

Infrastructure

Integrations

API Keys

JWT Keys

Log Drains

Add-ons

INTEGRATIONS

Data API

Vault

BILLING

Subscription

Usage

API Keys

Configure API keys to securely control access to your project

Publishable and secret API keys

Legacy anon, service_role API keys

Your new API keys are here

We've updated our API keys to better support your application needs. [Join the discussion on GitHub](#)

Having trouble with the new API keys? [Contact support](#)

Publishable key

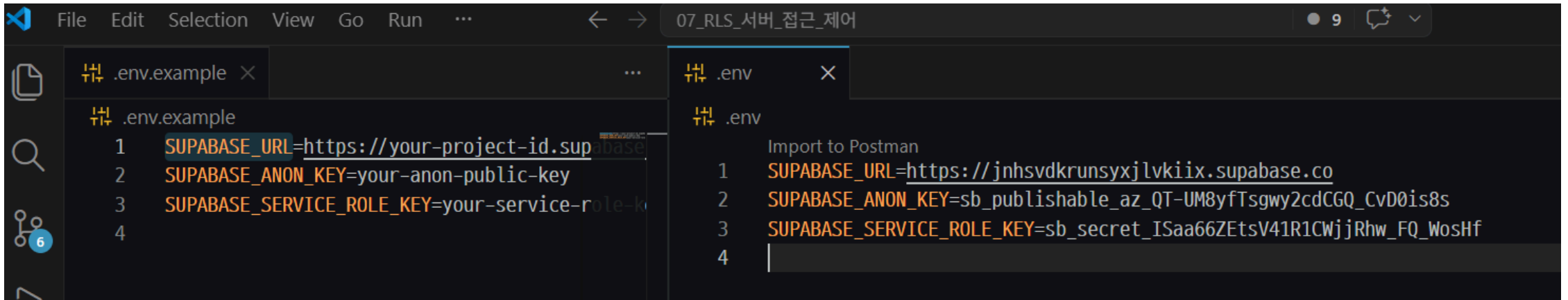
This key is safe to use in a browser if you have enabled Row Level Security (RLS) for your tables and configured policies.

NAME	API KEY
default	sb_publishable_az_UM8yfTsgwy2cdCGQ_CvD0i...

No description

Publishable keys can be safely shared publicly

복사한 key를 SUPABASE_ANON_KEY 에 추가



```
File Edit Selection View Go Run ... 07_RLS_서버_접근_제어 9
```

```
.env.example
1 SUPABASE_URL=https://your-project-id.supabase.co
2 SUPABASE_ANON_KEY=your-anon-public-key
3 SUPABASE_SERVICE_ROLE_KEY=your-service-role-key
4
```

```
.env
Import to Postman
1 SUPABASE_URL=https://jnhsvdkrunsyxjlvkiix.supabase.co
2 SUPABASE_ANON_KEY=sb_publishable_az_QT-UM8yftsgwy2cdCGQ_CvD0is8s
3 SUPABASE_SERVICE_ROLE_KEY=sb_secret_ISaa66ZetsV41R1CWjjRhw_FQ_WosHf
4
```

가상환경 생성

```
# pyproject.toml과 uv.lock 기준으로 의존성 설치  
uv sync
```

서버 실행

```
# 1. .env 파일에 실제 키 입력  
# 2. 서버 실행  
uvicorn main:app --reload --port 8001
```

Swagger UI 실습 (<http://localhost:8001/docs>)

각 API에서 Try it out을 누른 뒤 아래 순서대로 입력합니다.

이번 실습에서는 같은 계정 정보와 로그인 응답의 토큰을 다음 요청에서 계속 사용합니다.

이름	저장할 값
TEST_EMAIL	회원가입에 사용할 이메일
PASSWORD	회원가입에 사용할 비밀번호
ACCESS_TOKEN	로그인 응답의 <code>access_token</code>

실습 1. 회원가입

POST /auth/signup → Try it out → Body 입력 → Execute

```
{  
  "email": "swagger06@example.com",  
  "password": "swagger1234",  
  "display_name": "스웨거 학생"  
}
```

- 200 OK 확인
- 응답의 user_id, email, token_type 확인
- email_confirm_required 가 false 인지 확인

POST

/auth/signup Signup

^

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "email": "swagger06@example.com",
  "password": "swagger1234",
  "display_name": "스웨거 학생"
}
```

Execute

`access_token` 이 `null` 이고 `email_confirm_required` 가 `true` 라면 이메일 인증이 커진 상태입니다.

개발 실습에서는 Supabase의 **Authentication → Sign In / Providers → Email → Confirm email**을 끈 뒤 새 이메일로 다시 가입합니다.

[illegible]

실습 2. 잘못된 이메일 형식으로 422 확인

POST /auth/signup → Body 입력 → Execute

```
{  
  "email": "not-an-email",  
  "password": "swagger1234",  
  "display_name": "스웨거 학생"  
}
```

- 422 Unprocessable Entity 확인
- 오류의 loc 에 "body", "email" 이 표시되는지 확인

EmailStr 이 엔드포인트 실행 전에 이메일 형식을 검증합니다.

POST

/auth/signup Signup

^

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{  "email": "not-an-email",  "password": "swagger1234",  "display_name": "스웨거 학생"}

```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8001/auth/signup' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "not-an-email",
    "password": "swagger1234",
    "display_name": "스웨거 학생"
  }'
```

Request URL

`http://localhost:8001/auth/signup`

Server response

Code	Details
------	---------

422	Error: Unprocessable Entity
-----	-----------------------------

Response body

```
{
  "detail": [
    {
      "type": "value_error",
      "loc": [
        "body",
        "email"
      ],
      "msg": "value is not a valid email address: An email address must have an @-sign.",
      "input": "not-an-email",
      "ctx": {
        "reason": "An email address must have an @-sign."
      }
    }
  ]
}
```



Download

Response headers

```
content-length: 220
content-type: application/json
date: Sun, 21 Jun 2026 08:19:15 GMT
server: uvicorn
```

실습 3. 짧은 비밀번호로 422 확인

POST /auth/signup → Body 입력 → Execute

```
{  
  "email": "swagger06-short@example.com",  
  "password": "12345",  
  "display_name": "스웨거 학생"  
}
```

→ 422 Unprocessable Entity 확인

→ 오류의 loc 에 "body", "password" 가 표시되는지 확인

Field(..., min_length=6, max_length=100) 이므로 비밀번호는 6자 이상이어야 합니다.

POST

/auth/signup Signup

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{  "email": "swagger06-short@example.com",  "password": "12345",  "display_name": "스웨거 학생"}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8001/auth/signup' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "swagger06-short@example.com",
    "password": "12345",
    "display_name": "스웨거 학생"
  }'
```

Request URL

`http://localhost:8001/auth/signup`

Server response

Code

Details

422

Error: Unprocessable Entity

Response body

```
{
  "detail": [
    {
      "type": "string_too_short",
      "loc": [
        "body",
        "password"
      ],
      "msg": "String should have at least 6 characters",
      "input": "12345",
      "ctx": {
        "min_length": 6
      }
    }
  ]
}
```



Download

Response headers

```
content-length: 154
content-type: application/json
date: Sun, 21 Jun 2026 08:21:18 GMT
server: uvicorn
```

실습 4. 잘못된 비밀번호로 로그인 실패

POST /auth/login → Body 입력 → Execute

```
{  
  "email": "swagger06@example.com",  
  "password": "wrong1234"  
}
```

→ 401 Unauthorized 확인

```
{  
  "detail": "로그인에 실패했습니다"  
}
```

POST

/auth/login

Login

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

```
{  "email": "swagger06@example.com",  "password": "wrong1234"}  
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8001/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "swagger06@example.com",
    "password": "wrong1234"
  }'
```

Request URL

`http://localhost:8001/auth/login`

Server response

Code

Details

401
Undocumented

Error: Unauthorized

Response body

```
{
  "detail": "로그인에 실패했습니다"
}
```



Download

Response headers

```
content-length: 44
content-type: application/json
date: Sun, 21 Jun 2026 08:23:14 GMT
server: uvicorn
```

실습 5. 로그인하고 access_token 저장

POST /auth/login → 회원가입 때 사용한 이메일과 비밀번호 입력 → Execute

```
{  
  "email": "swagger06@example.com",  
  "password": "swagger1234"  
}
```

→ 200 OK 확인

→ 응답의 access_token 값을 ACCESS_TOKEN 으로 복사

→ 회원가입 응답과 로그인 응답의 user_id 가 같은지 확인

이후 인증 요청에는 refresh_token 이 아니라 access_token 을 사용합니다.

POST

/auth/login

Login

^

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

▼

Edit Value

Schema

```
{  "email": "swagger06@example.com",  "password": "swagger1234"}  
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8001/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "swagger06@example.com",
    "password": "swagger1234"
  }'
```



Request URL

http://localhost:8001/auth/login

Server response

Code	Details
------	---------

200

Response body

[illegible]

Download

Response headers

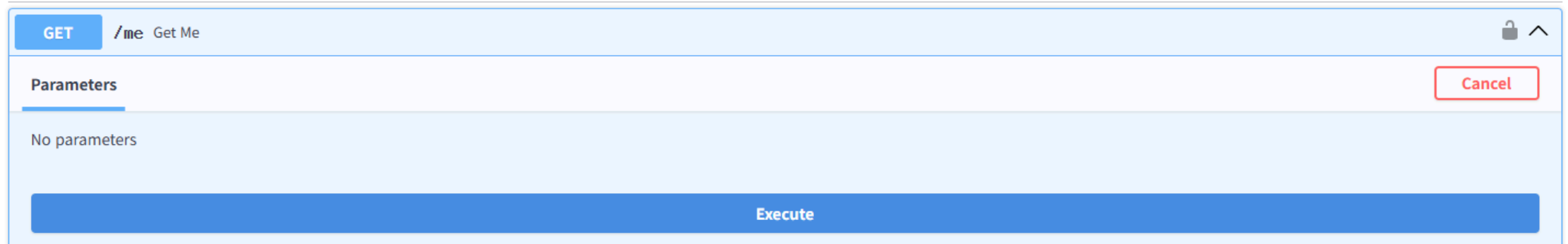
```
content-length: 1118
content-type: application/json
date: Sun, 21 Jun 2026 08:24:51 GMT
server: uvicorn
```

실습 6. 토큰 없이 보호된 API 요청

아직 **Authorize**를 실행하지 않은 상태에서 `GET /me` → Try it out → Execute

→ `403 Forbidden` 확인

```
{  
  "detail": "Not authenticated"  
}
```



The screenshot shows an API client interface. At the top, there's a header bar with a blue button labeled 'GET' and the text '/me Get Me'. To the right of this bar are a lock icon and an upward arrow icon. Below the header, there's a section titled 'Parameters' with a red 'Cancel' button to its right. Under 'Parameters', it says 'No parameters'. At the bottom of the interface is a large blue button labeled 'Execute'.

이미 인증했다면 우측 상단 Authorize → Logout → Close로 토큰을 지운 뒤 실행합니다.

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8001/me' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8001/me
```

Server response

Code	Details
403 <i>Undocumented</i>	Error: Forbidden

Response body

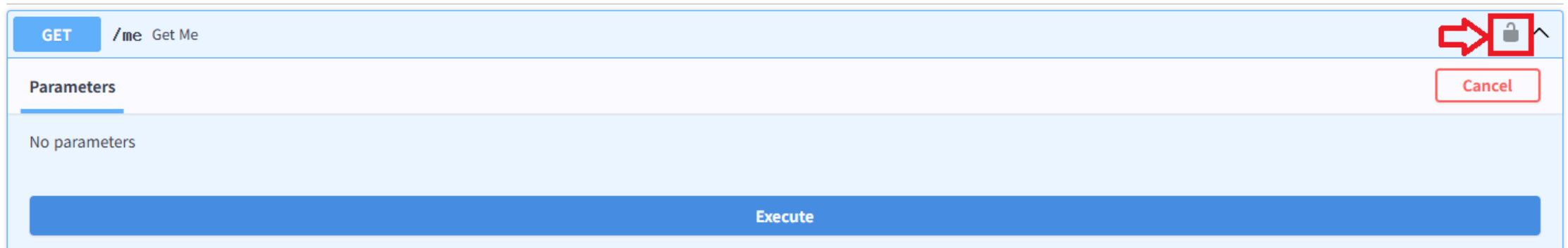
```
{
  "detail": "Not authenticated"
}
```

Response headers

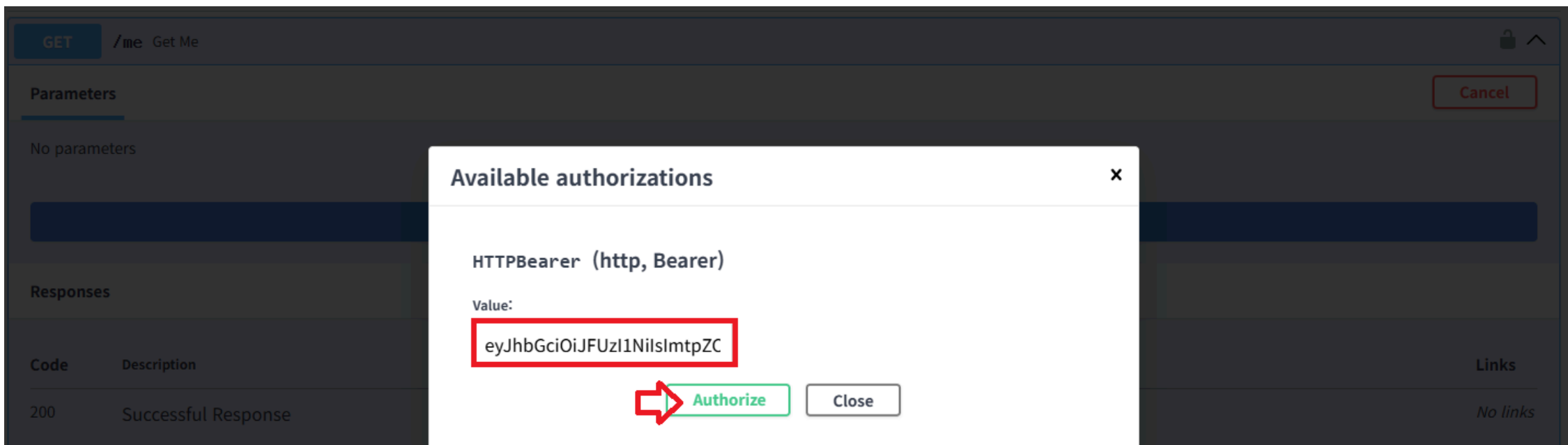
```
content-length: 30
content-type: application/json
date: Sun, 21 Jun 2026 08:26:37 GMT
server: uvicorn
```

실습 7. Bearer 토큰으로 현재 사용자 조회

1. Swagger UI 우측 상단 Authorize 클릭

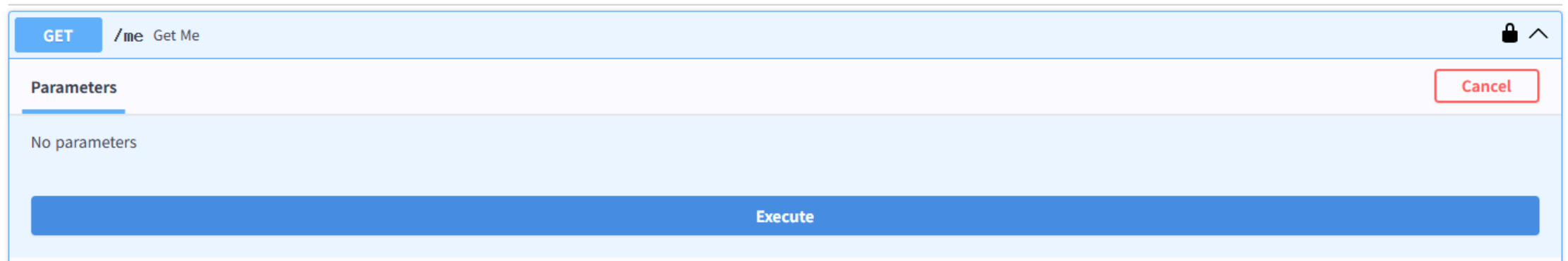


2. Value에 ACCESS_TOKEN 값만 입력



3. GET /me → Try it out → Execute

→ 200 OK 와 다음 형태의 응답 확인



The image shows a REST client interface. At the top, there is a header bar with a blue button labeled 'GET' and a text input field containing '/me Get Me'. To the right of the input field are a lock icon and an upward arrow icon. Below the header bar is a section titled 'Parameters' with a red 'Cancel' button in the top right corner. The main area of the 'Parameters' section contains the text 'No parameters'. At the bottom of the interface is a large blue button labeled 'Execute'.

→ /me 응답의 user_id 와 로그인 응답의 user_id 가 같은지 확인

Swagger UI가 요청 헤더에 `Authorization: Bearer {ACCESS_TOKEN}` 을 자동으로 추가합니다.

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8001/me' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsImtpZCI6IjM2NTVhOGMwLTl0YjctNDVmNi04YzUzLTNmNWMM3NjkwNTA2YiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwczovL21wa3pncXdpYmhiZ3NobHVhYWNjLnN1cGFiYXN1LmNvL2F1dGgvdjEiLCJzdBWl...
```

Request URL

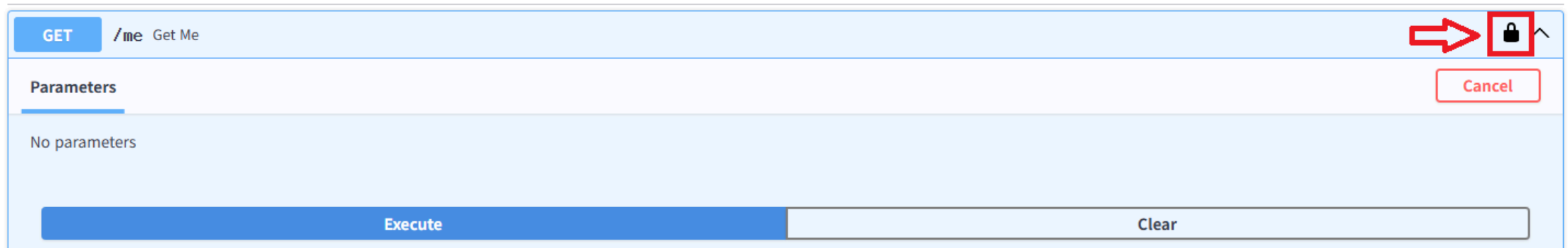
```
http://localhost:8001/me
```

Server response

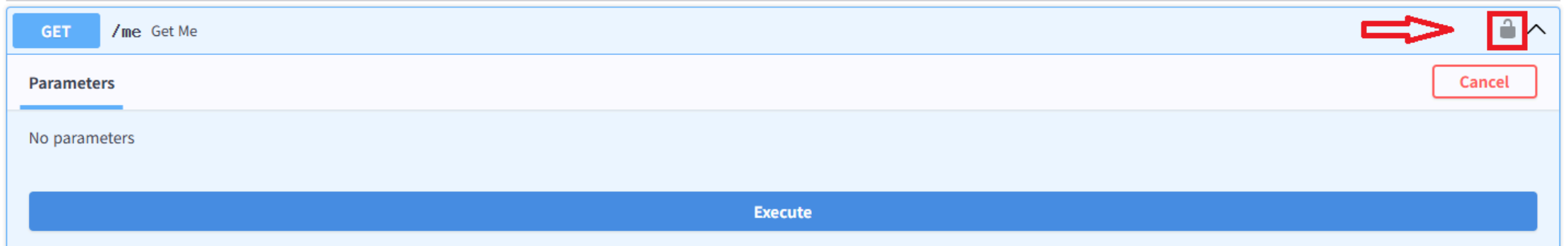
Code	Details
200	Response body <pre>{ "user_id": "d14e5576-ff8d-4b66-b862-1b82bcbeb983", "email": "swagger06@example.com", "message": "인증 성공! 이 정보는 토큰에서 읽었습니다." }</pre> Download
	Response headers <pre>content-length: 153 content-type: application/json date: Sun, 21 Jun 2026 08:30:22 GMT server: uvicorn</pre>

실습 8. 유효하지 않은 토큰으로 401 확인

1. 우측 상단 Authorize → Logout → Close로 기존 토큰 제거



2. 다시 **Authorize**를 열고 Value에 `invalid-token` 입력



GET /me Get Me

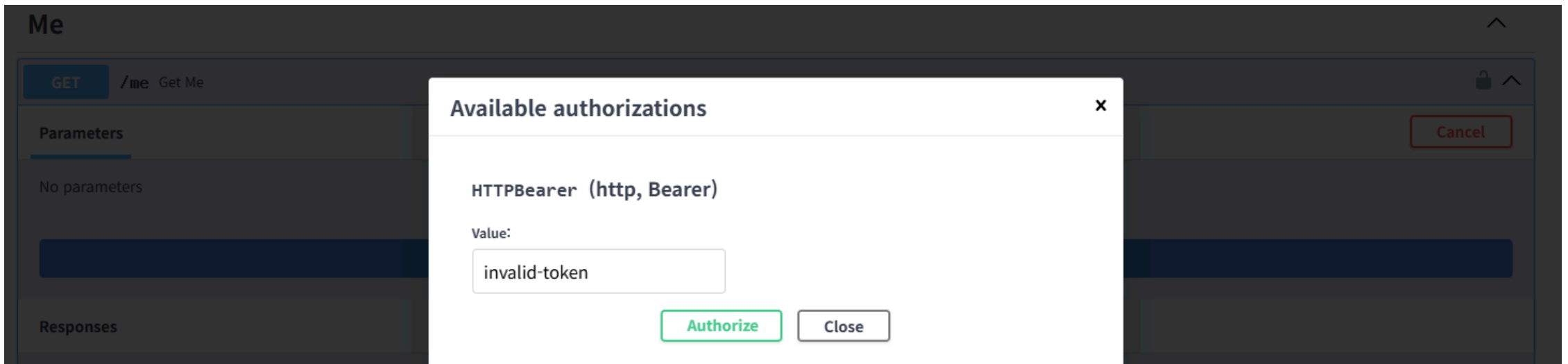
Parameters

No parameters

Execute

Cancel

3. Authorize → Close 클릭



4. GET /me 실행

GET

/me Get Me

🔒 ^

Parameters

Cancel

No parameters

Execute

→ 401 Unauthorized 확인

토큰이 없는 요청은 HTTPBearer 가 403 으로 거부하고, 형식은 있지만 검증에 실패한 토큰은 get_current_user() 가 401 로 거부합니다.

Code	Details
401 <i>Undocumented</i>	Error: Unauthorized Response body <pre>{ "detail": "유효하지 않은 토큰입니다" }</pre> Response headers <pre>content-length: 48 content-type: application/json date: Sun, 21 Jun 2026 08:43:14 GMT server: uvicorn</pre>